

GITHUB LINK:

https://github.com/mingyaoma8/Clasher_Project.git

1. Project Goals and Planned Data Collection

The main goal of our project was to figure out which Clash Royale deck type performs the best, where a deck type is defined by its main win-condition card (such as Hog Rider, Golem, or Miner). To do this, we planned to use the official Clash Royale API along with the WorldTime API.

We planned to collect data from real users' gaming IDs, which allowed us to access actual battle histories and decks that real players used in their matches. From the Clash Royale API, we aimed to gather card data, battle results, crowns scored, and full deck compositions for each battle. This would allow us to identify a deck's key card and calculate win rates for different deck types.

We also planned to use the WorldTime API to support time-based analysis. By combining battle timestamps with time and weekday information, we wanted to explore whether factors such as time of day affected performance. All of this data was planned to be stored in a SQLite database and analyzed using SQL joins, calculations, and visualizations.

2. Goals Achieved and Data Collected

The goals that were achieved including what APIs/websites you actually worked with and what data you did gather (5 points)

We successfully achieved the main goals of the project. We utilized two external APIs: the official Clash Royale API and the WorldTime API.

From the Clash Royale API, we collected battle data from real players, resulting in a dataset of 115 unique battle logs (surpassing the 100-row minimum). This data included card information, battle outcomes (win/loss), crowns scored, and full deck compositions, stored in relational tables (cards, battles, and battle_cards).

Crucially, we also successfully integrated the WorldTime API, collecting 105 entries of time-specific data (also surpassing the 100-row minimum) and storing them in the timeinfo table.

This complete dataset was stored in an SQLite database, allowing us to use SQL joins to perform complex analysis, such as identifying key win-condition cards and calculating win rates. Finally, we used Matplotlib to create the required visualizations, including the

win rate by deck type, the stacked bar chart of wins versus losses per deck, and the bar chart of win rate by time of day. We were successfully able to analyze real user data and draw insights from actual gameplay patterns.

3. Problems Encountered

3. The problems that you faced (5 points)

One of the biggest challenges we faced was making sure that all derived fields were correctly populated when inserting data into the database. In early versions of our script, fields such as `deck_key_card` and `time_of_day` were missing, which caused analysis queries to return empty results even though the tables themselves existed.

Another challenge was working with the Clash Royale battleTime format, which does not follow a standard datetime format. This caused issues when extracting weekday and hour information and required manual parsing to fix.

We also encountered confusion due to having multiple database files during development, which made it unclear which database contained the correct tables and data. Once we standardized all scripts to use a single database file, this problem was resolved.

4. Calculations from the Data in the Database

The calculations from the data in the database (i.e. a screen shot) (5 points)

To analyze our data, we performed several calculations using SQL queries on our SQLite database.

First, we calculated win rate by deck type, where each deck type is defined by its key win-condition card (`deck_key_card`). For each deck type, we calculated:

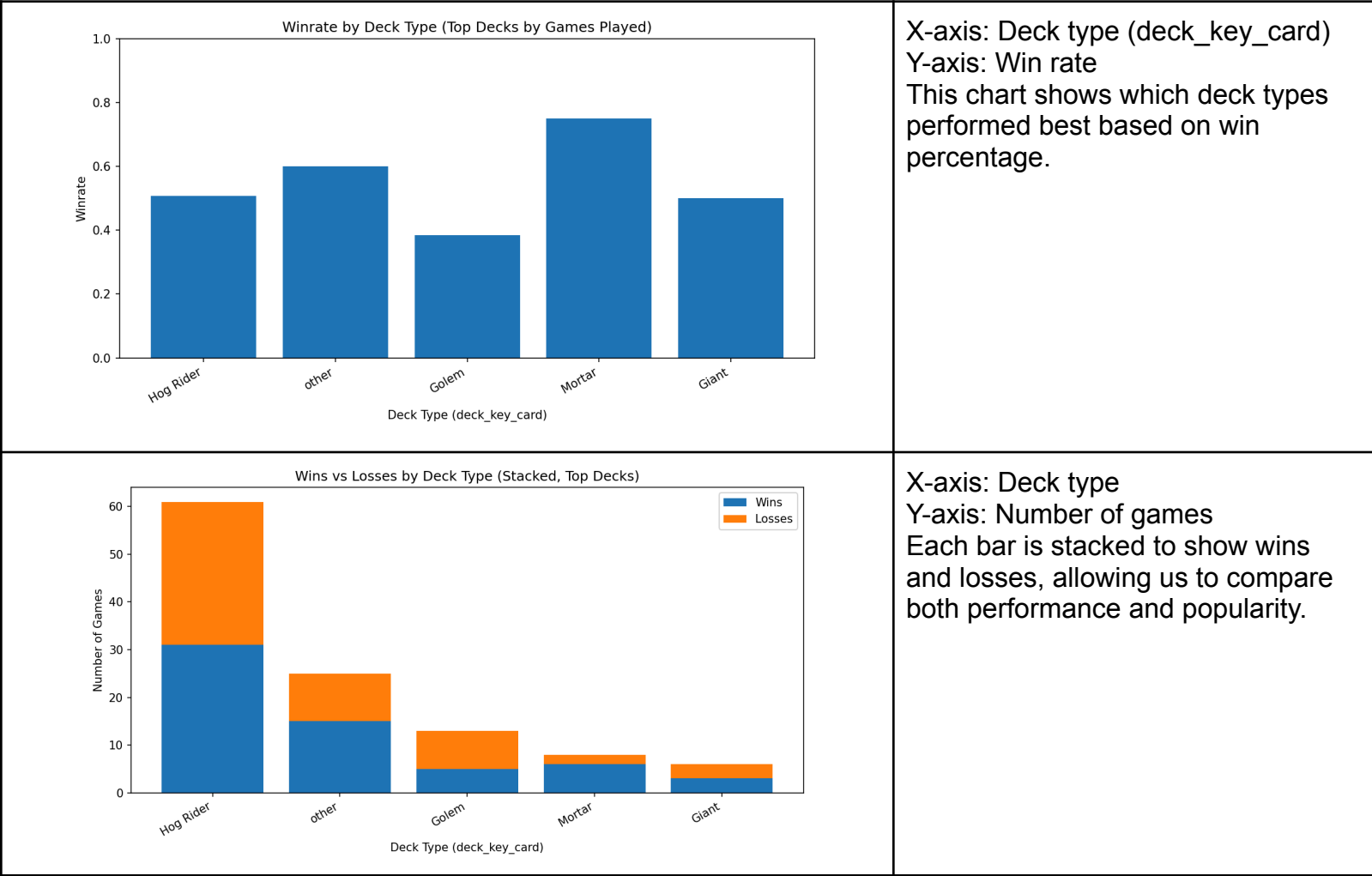
- total number of games played
- number of wins
- win rate = wins ÷ total games

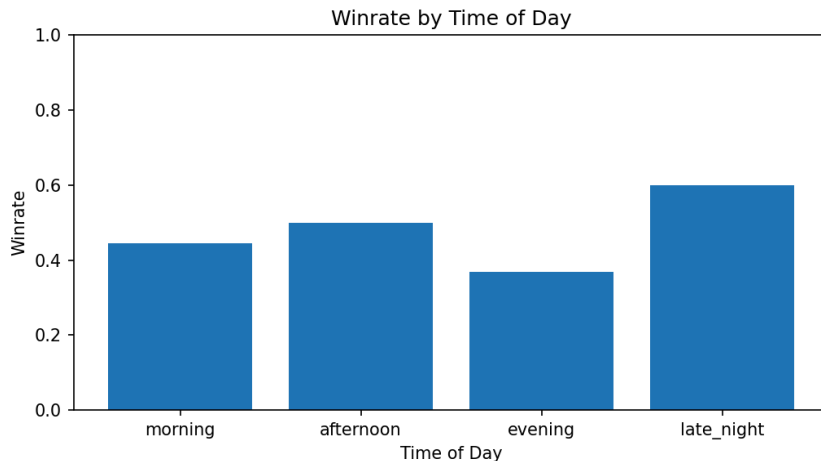
We also calculated wins and losses per deck type, which allowed us to visualize how often each deck won or lost relative to how frequently it was used.

In addition, we calculated win rate by time of day using the derived `time_of_day` field (morning, afternoon, evening, late_night). This allowed us to compare player performance across different time periods.

These calculations were performed directly from the battles table, and in some cases joined with other tables (such as battle_cards and cards) to support deeper analysis. A screenshot of the calculated outputs is included above.

5. The Visualizations





X-axis: Morning, Afternoon, Evening, Late Night

Y-axis: Win rate

This visualization shows how performance changes depending on when games are played.

3. Instructions for Running Code

Prerequisite

1. Install the required Python libraries (requests, matplotlib, etc.) by running the following command in your terminal: `pip install requests matplotlib`

Setup and Data Collection

1. Clone Repository: Clone the project from GitHub and change your directory to the project folder.
 - Command: `git clone [Your Repository URL Here]`
 - Command: `cd Final Project Clashers`
2. Database Setup: Run the script to create the four initial tables (cards, battles, battle_cards, timeinfo) in the clash_data.db database.
 - Command: `python database_setup.py`
3. Collect Clash Royale Data: Run the main data collection script. This will connect to the Clash Royale API and populate the cards, battles, and battle_cards tables with over 100 battle logs.
 - Command: `python clash_data.py`
4. Collect WorldTime API Data: Run the time API script multiple times (at least 5 times) to collect enough data and ensure the timeinfo table has over 100 rows, accounting for potential connection resets during collection.
 - Command (repeat 5 times): `python timeapi_data.py`

Analysis and Visualization

1. Run Analysis and Plotting: Execute the script that performs the final SQL calculations, prints the statistics, and generates the required PNG visualization files (winrate_by_deck_type.png, deck_wins_losses.png, and time_of_day_winrates.png).
 - Command: `python visualize_data.py`

The final charts (PNG files) and the statistical output (in a text file) will be saved directly within the project directory.

7. Function Diagram

database_setup.py

```
create_tables(conn)
  > Input: SQLite connection
  > Output: None
  > Creates tables: cards, battles,
    battle_cards, timeinfo
```



clash_data.py

Ming Yao Ma

```
fetch_and_store_cards(conn)
  > Input: conn
  > Output: # cards inserted
  > Calls /v1/cards API

fetch_and_store_battles(conn, player_tag, limit)
  > Input: conn, player_tag, limit ≤ 25
  > Output: # battles inserted
  > Calls /v1/players/{tag}/battlelog
  > Inserts battles + their cards
  > Derives: result, deck_key_card, time_of_day
```

writes to database



timeapi_data.py

Jessica Park

```
fetch_and_store_timeinfo(conn, limit)
  > Input: conn, limit ≤ 25
  > Output: # time rows inserted
  > Calls WorldTimeAPI
  > Stores: datetime, hour, weekday
```

writes to database



analysis_and_viz.py

Both

```
calculate_deck_winrates(conn)
  > Returns: stats per deck (games, wins, winrate)

calculate_time_of_day_winrates(conn)
  > Returns: winrate by time_of_day

plot_deck_winrates(data)
  > Bar chart: deck type → winrate

plot_deck_wins_losses(data)
  > Stacked chart: wins/losses per deck

plot_time_of_day_winrates(data)
  > Bar chart: morning/afternoon/evening/night
```



Final Visualizations
(PNG files saved & added to report)

8. You must also clearly document all resources you used. The documentation should be of the following form (10 points)

Date: 12/1/25

Issue Description: Parsing Clash Royale battletime format

Location of Resource: Clash Royale API

Result (did it solve the issue?): Learned correct datetime parsing, solved the issue.

Date: 12/4/25

Issue Description: SQL JOIN syntax for grouped win rates

Location of Resource: SQLite documentation

Result (did it solve the issue?): Correct GROUP BY and JOIN queries, solved the issue.

Date: 12/5/25

Issue Description: Matplotlib stacked bar charts

Location of Resource: Matplotlib official docs

Result (did it solve the issue?): Implemented wins vs losses chart. Solved issue.

Date: 12/10/25

Issue Description: Debugging missing columns

Location of Resource: SQLite PRAGMA docs

Result (did it solve the issue?): Identified schema mismatches. Solved issue.

Date: 12/10/25

Issue Description: Pandas SQL integration

Location of Resource: Pandas documentations

Result (did it solve the issue?): Used read_sql_query successfully. Solved issue.

Date: 12/15/25

Issue Description: Handling ConnectionResetError

Location of Resource: Python requests library documentation

Result (did it solve the issue?): Implemented retry logic (multiple attempts) to successfully gather over 100 rows, solving the intermittent connection issue.